Forest Timber Traceability Blockchain

A local proof-of-concept for timber identity and chain-of-custody tracking

PROJECT PURPOSE

The application provides a digital workflow for assigning an identity to each timber log (grume) and recording successive chain-of-custody events against that identity. Its purpose is to make timber records and their history easier to consult and verify through a web interface and a tamper-evident, hash-linked ledger.

HOW THE SYSTEM IS ORGANIZED

Frontend — Python Panel presents the application screens and user workflows, including timber registration, traceability consultation and ledger inspection.

Backend — JavaScript with Node.js and Express exposes the REST API used by the interface to submit and retrieve timber records and events.

Ledger — records are stored in persistent hash-linked blocks in data/forest-blockchain.json. SHA-256 links support tamper-evident checks and ledger verification.

Integration boundary — the REST API is the intended connection point for an OpenForis Arena adapter.

TIMBER IDENTITY & REGISTRATION

The registration form captures timber attributes such as species and forest origin. The README identifies dropdown options including Akon, Sapelli, Lobaye and Berberati.

Volume is calculated from diameter and length using $V = \pi \times (D / 2)^2 \times L$. Diameter is entered in centimetres and length in metres; the calculated volume is read-only.

The harvest date is submitted in dd-mm-yyyy format. Browser geolocation can capture coordinates with the “Get my location” action; manual GPS entry is available if permission is denied.

CHAIN OF CUSTODY & API

A timber record can be associated with later events, for example a TRANSPORT event containing the grume ID, event type, actor, location and notes. This links operational activity to a specific timber identity.

The API includes health checking, chain and timber retrieval, timber registration, event submission, per-grume history and ledger verification. Protected endpoints require the x-api-key header.

This structure provides a consistent interface for the Panel application and a future external system adapter.

OPENFORIS ARENA INTEGRATION

The backend currently exposes an integration boundary; it does not claim that live Arena synchronization is already active. A working adapter still requires the Arena instance URL, authentication method, relevant data model or table names, and permission to call the Arena API.

PROTOTYPE SCOPE & PRODUCTION READINESS

This is a single-node development blockchain prototype. Hash linking and verification make changes detectable within the stored chain, but the system is not a decentralized consensus network and does not provide independent validators, distributed replication or production-grade immutability.

Before shared or production deployment, the README recommends transactional database storage, authenticated users and role-based permissions, TLS and secret management, audit logging, signed transactions and key management, backups, monitoring, rate limits, schema validation, automated tests, and validation with the relevant forestry authority.

Descriptive overview based on the supplied project README.

3. Chain of custody, traceability and blockchain explorer

Record a traceability event

The event form associates a Grume ID with an event type, responsible operator/organization, location and notes. The configured event types are HARVEST, FOREST_GATE, TRANSPORT, SAWMILL_INTAKE, PROCESSING, WAREHOUSE, SALE and EXPORT. Submitting the form sends a POST request to `/events` and displays the API response.

Traceability history

The history screen accepts a Grume ID and requests `/timber/{Grume ID}/history`. On success, it displays the returned history in a paginated table. This supports reviewing the chain-of-custody events recorded for a selected timber item.

Blockchain explorer and verification

The Verify entire blockchain button calls `/ledger/verify`. The result is presented as a success or error alert according to the API's `valid` field. The Load blocks button calls `/chain` and renders each block as an expandable JSON pane. The dashboard block table also exposes shortened previous-hash and hash values, plus the nonce.

Blockchain field	Meaning in the application display
Index	Block sequence number.
Timestamp	Time associated with the block.
Transactions	Number of transactions stored in the block.
Previous hash	Truncated previous-block hash shown in the dashboard table.
Hash	Truncated current-block hash shown in the dashboard table.
Nonce	Proof-of-work/consensus nonce field returned by the API.

Operational note

The user interface relies on the backend to enforce blockchain rules, uniqueness and transaction processing. The uploaded Python file shows the API requests and the presentation logic; it does not contain the Node.js backend implementation or the full consensus/validation rules.

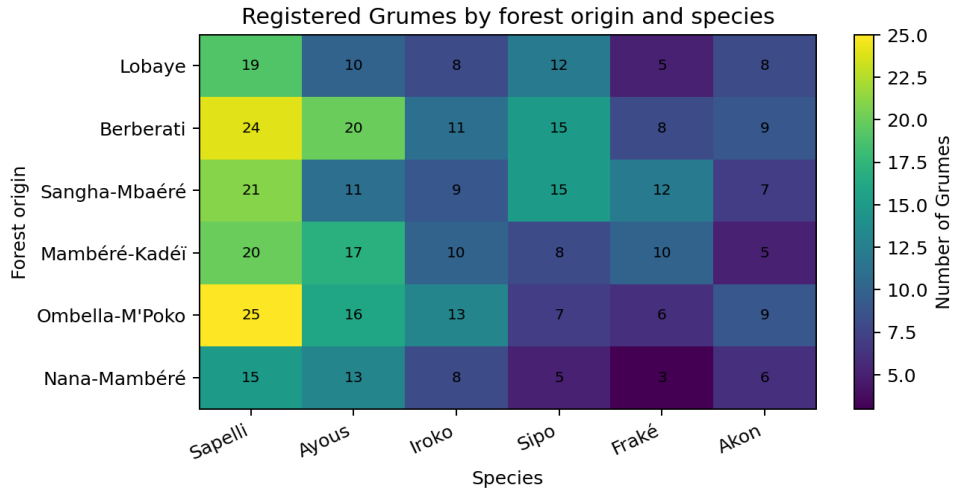
4. Statistics graph dashboard

The Statistics graph screen reads timber registration data from `data/forest-blockchain.json`. It supports blockchain exports where registrations are recorded in block transactions of type `TIMBER_REGISTERED`, and also supports a top-level timber dictionary. The Essence and Origine dropdowns filter all displayed analytics. Reload JSON data rereads the file.

KPI	Interpretation
Total Grumes	Number of records after applying the selected filters.
Total volume	Sum of Volume (m³) for the filtered records.
Essences	Number of distinct species in the filtered records.
Origins	Number of distinct forest origins in the filtered records.

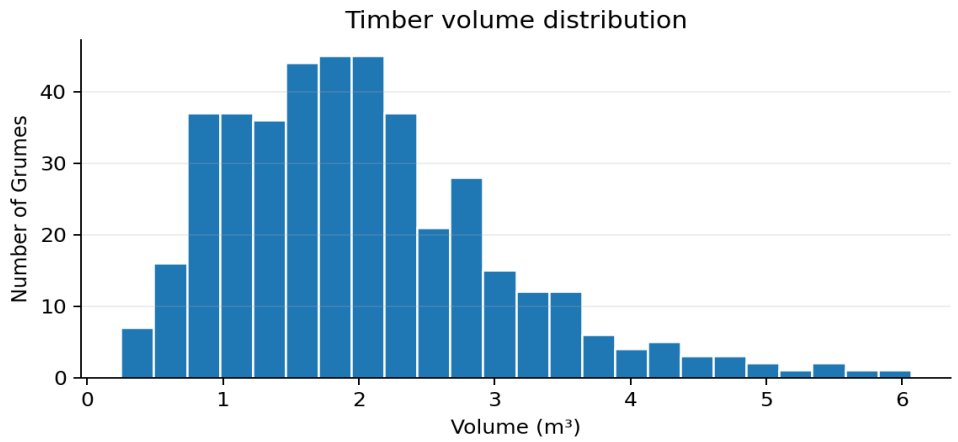
What this view shows

Heatmap — rows represent forest origins and columns represent timber species. Each cell shows the number of registered Grumes for that origin/species combination. Stronger color indicates a larger count. To keep the chart readable, the app limits the view to the 15 most represented species and 15 most represented origins.



What this view shows

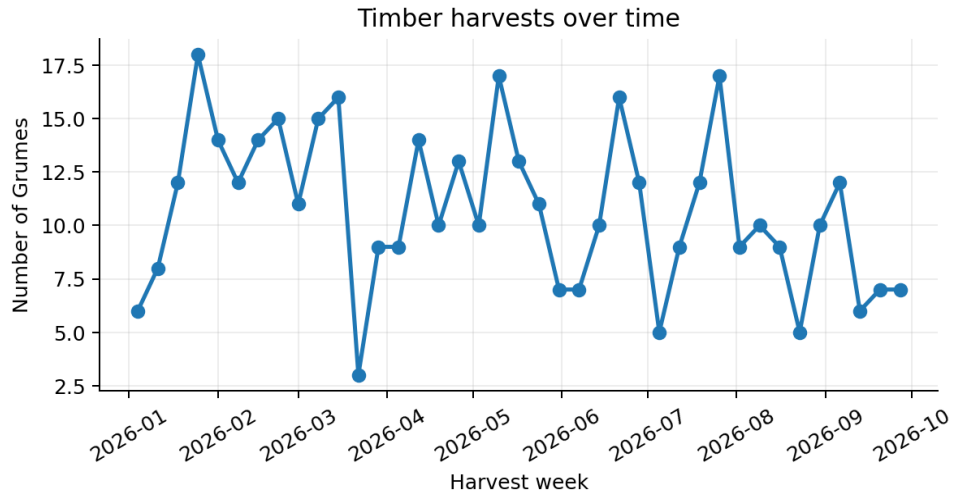
Histogram — shows the distribution of registered timber volume in cubic metres, using 25 bins. The horizontal axis is volume range and the vertical axis is the number of Grumes in each range. It helps identify common volume ranges, spread and unusually high or low values.



4. Statistics graph dashboard (continued)

What this view shows

Line chart — groups records by harvest date and plots the number of Grumes for each date, sorted chronologically. Peaks indicate dates with more registered harvests. The graph is interactive in the app and includes hover, pan, zoom, reset and save tools.



Summary table

The table groups filtered records by Essence and calculates the number of Grumes, total volume, average volume, average diameter and average length. Numeric results are rounded to two decimal places. It uses a paginated Tabulator table with header filters, allowing users to inspect species categories.

Species	Grumes	Total m³	Avg m³	Avg dia. cm	Avg length m
Sapelli	124	253.51	2.04	62.6	6.3
Ayous	87	168.80	1.94	61.1	6.3
Sipo	62	113.86	1.84	60.5	6.1
Iroko	59	119.25	2.02	62.9	6.1
Akon	44	93.12	2.12	63.7	6.2
Fraké	44	99.52	2.26	65.6	6.5

The table above is a sample presentation based on the demonstration records used to draw this report's charts. The live app will calculate the same columns from its JSON data.

5. OpenForis Arena integration and technical configuration

The Arena integration screen provides fields for an Arena API base URL and an access token. The code explicitly states that credentials are not sent to Arena until a dedicated Arena adapter is configured. The current interface therefore documents/configures the intended connection but does not implement a complete Arena synchronization workflow.

Configuration and dependencies visible in the code

Item	Configuration / role
Frontend	Python Panel application using Panel widgets, cards, tabs and a FastListTemplate.
Analytics	Pandas and hvPlot; statistics read the local blockchain JSON file.
Geospatial display	Folium map embedded in a Panel HTML pane; Branca adds the origin legend.
API client	Python requests library; GET/POST requests use configurable API URL and optional API key.
Excel	Pandas Excel reader/writer and in-memory BytesIO for import/export.
API timeout	GET requests: 4 seconds; POST requests: 8 seconds.
JSON path	data/forest-blockchain.json in a data folder beside app.py.

Run and deployment checklist

- Install the Python dependencies used by the application: panel, pandas, requests, hvplot, folium, branca and openpyxl.
- Start the Node.js blockchain API and confirm that its health endpoint is reachable.
- Set BLOCKCHAIN_API_URL if the API is not at the default localhost address; configure API_KEY if authentication is enabled.
- Place forest-blockchain.json in a data folder beside app.py for the Statistics graph tab.
- Run the Panel server, for example: panel serve app.py --port 5006 --show --autoreload.
- Use localhost or HTTPS when browser geolocation is required, and grant location permission in the browser.
- Configure a dedicated Arena adapter before expecting data to synchronize with OpenForis Arena.

Source and scope

This document is based on the uploaded app.py source file. It describes the interface and behaviour implemented in that file. It does not verify the live API, inspect the actual blockchain database, or validate the underlying Node.js blockchain service. Charts and sample summary values are illustrative because no live JSON dataset was supplied.